

Section 1 — Product Architecture: Adversarial Review & Hardening

Scope (per user redirect): Focus on **Section 1 (Product Architecture)** only. Adversarially review it in isolation from many angles, harden it, then organize the result and commit it to git.

Context

The original Section 1 sketched a single-purpose, "press record and walk away" multitrack recorder with four pillars (Trust, Speed, Awareness, Recoverability), three personas, a four-mode flow, and a short non-goals list. Treated in isolation and attacked from multiple directions, it holds up directionally but has **material gaps** that would bite a real broadcast/reality-TV deployment. This document records the attack, then a hardened Section 1, then how it lands in git.

Part A — Adversarial Review (by angle)

Each finding: **severity** (Critical / High / Medium / Low) + the fix folded into Part B.

A1 — Competitive positioning (High)

Original states "inspired by Boom Recorder" but never says *why someone picks this over the alternatives*: Boom Recorder, Reaper-as-recorder, Metacorder, Sound Devices Wave Agent (metadata), or a hardware recorder (Sound Devices/Cantar/Aaton/Nomad). Without an explicit wedge, the product is "another recorder." **Fix:** add a positioning statement — software-defined, high-density (Dante-class) iso recorder with first-class metadata/naming/post hand-off and operational awareness, on commodity Apple Silicon. Differentiator = reliability + workflow + scale at software cost, not yet-another-feature-set.

A2 — The headline promise overclaims (Critical)

"Press record, walk away 12 hours, get perfect files" is **unfalsifiable and unsafe** as stated. A single Mac mini faces power loss, kernel panics, thermal throttle, OS auto-updates, sleep/App Nap, Dante network blips, disk-full. The promise is only credible when paired with explicit *system-level* dependencies and *measurable guarantees*. **Fix:** (1) replace the vague promise with **explicit product guarantees/SLOs** (below); (2) add a **System Reliability Dependencies** subsection (appliance posture, sleep/App Nap inhibition, OS-update/Time-Machine policy, UPS expectation, NTP policy).

A3 — Missing reliability dependencies the app itself must own (Critical)

The app must actively defend its own runtime: prevent **App Nap** and **system/display sleep** while armed/recording, refuse cloud/network/Time-Machine destinations (already noted for §7 but it's a *product* principle), and warn on low power / thermal pressure. None of this was in Section 1. **Fix:** new subsection "System Reliability Dependencies" + elevate "Capture is sacred" to a first-class principle.

A4 — Take-based vs. continuous paradigm tension (High)

Section 1 implicitly leans **film/scripted** (take/roll). Reality TV, live events, and broadcast are predominantly **continuous iso recording** with marker-based logging and rollover, not discrete takes. The product targets *both*, but Section 1 never names the tension — which would misshape the whole UX and data model. **Fix:** state both operating paradigms explicitly (Take mode vs Continuous mode) as a product-level concept.

A5 — Pre-record buffer is absent (High)

Every serious production recorder (Boom Recorder, Sound Devices) offers a **pre-record / pre-roll buffer** (capture N seconds *before* the operator hits record). Omitting it from the product mode model is a competitive and operational miss. **Fix:** add a **Standby (pre-record buffering)** state to the mode model.

A6 — No degraded/error state in the mode model (High)

The "if a subsystem fails, recording continues and is logged" philosophy has **no state to represent it**. Setup→Armed→Recording→Review can't express "Recording (Degraded): mirror lost / LTC lost / disk warning." **Fix:** add **Recording (Degraded)** as an explicit operating state and define the failure hierarchy.

A7 — Redundancy / single-point-of-failure (High)

"Walk away 12 hours" on one box ignores that broadcast/reality runs redundant recorders (A/B) and UPS. Section 1 doesn't state whether multi-machine redundancy is in product scope (even future). **Fix:** add a redundancy stance — mirrored disks (near-term, already in spec) + a documented "run a second instance as backup recorder" posture, with full A/B chase deferred but acknowledged.

A8 — Personas are incomplete and conflated (Medium)

"Sound mixer / A2" conflates roles; reality/live use a **central audio control room operator** and **sound utility**, plus the **rental/facility tech** who builds presets, the **producer/showrunner** who only wants the dashboard, and the **Dante/IT network admin**. Also: buyer (production company) ≠ user. **Fix:** expand and de-conflate personas; separate buyer from users.

A9 — Missing pillar: interoperability / post hand-off (Medium)

BWF/iXML/markers/CSV/daily-folders are a *core value* (post integration) but aren't reflected in the pillars. The pillar set under-weights "fits the production→post pipeline." **Fix:** add a fifth pillar — **Interoperability / clean hand-off to post**.

A10 — Confidence monitoring output is unaddressed (Medium, open decision)

"Input metering only" + spot-check playback omits **live confidence monitoring** (headphone/PFL/solo) that production-sound operators expect. This is a genuine product-scope decision, not an oversight to silently keep out. **Fix:** surface as an explicit **open product decision** (in-scope live monitoring vs. metering-only), with a recommendation.

A11 — "No mixer" vs. per-track gain inconsistency (Low)

Non-goals say "no mixer," but the product has per-track gain. Ambiguous. **Fix:** clarify gain is **record trim**, not a monitor mix bus.

A12 — System clock / time integrity (Medium)

Wall clock drives naming, daily folders, and midnight logic; NTP jumps / DST / timezone changes can corrupt names and confuse midnight detection (which §8 separates from LTC corruption). **Fix:** add a product-level time-integrity stance (recommend stable/NTP-disciplined-but-no-step clock during record; treat wall-clock and media-clock as distinct).

A13 — Appliance vs. general-purpose app (Medium)

Reliability is far higher if the target Mac mini runs as a **dedicated appliance** (auto-launch, auto-recover, minimal other software, kiosk-ish) rather than a general desktop app among many. This is a product-architecture choice that was never made. **Fix:** state the **appliance-first deployment posture** as the recommended operating model.

A14 — Accessibility / colorblind-safe metering / localization (Low)

"Modern UX" for international crews implies colorblind-safe meters, large-text legibility at a distance, and localization. Not mentioned. **Fix:** add a short UX-principles note at product level.

A15 — Guarantees are not measurable (Critical, ties A2)

"Trust" and "perfect files" can't gate development. Product architecture should declare testable guarantees that become acceptance gates. **Fix:** **Product Guarantees (SLOs)** subsection with falsifiable statements.

Part B — Hardened Section 1 (Product Architecture)

1.0 Positioning

A **software-defined, high-density isolated-track recorder** for Apple Silicon: Dante-class channel counts, first-class metadata/naming/post hand-off, and broadcast-grade operational awareness — at commodity-hardware cost. It competes with Boom Recorder/Metacorder/Reaper-as-recorder and offloads from hardware recorders where channel density, metadata flexibility, and dashboarding matter more than a self-contained device. **Wedge = reliability + workflow + scale, not feature count.**

1.1 What it is

A single-purpose, always-trustworthy recorder whose **prime directive is "Capture is sacred"**: the capture path must never drop a sample; *every other subsystem (UI, mirror, LTC, network, dashboard) is best-effort and may degrade, but never stops or corrupts capture.*

1.2 Product Guarantees (SLOs) — testable, become acceptance gates

- **G1 No-loss:** zero dropped samples under the supported load envelope (channel count × SR × bit depth on the certified device matrix).

- **G2 Crash-valid:** every record press yields a recoverable, openable file even on power loss / kill-9 mid-write.
- **G3 TC-accurate:** per-file start timecode and BWF `TimeReference` are sample-accurate when LTC is present.
- **G4 Degrade-don't-die:** any single non-capture subsystem failure leaves capture running and is logged + surfaced.
- **G5 Long-run-stable:** bounded memory/CPU/thermals across multi-hour and multi-day soak (no leaks, no drift-induced xruns).

1.3 Product Pillars

1. **Trust** — capture never glitches; guarantees above are gated by tests.
2. **Continuity / uptime** — built for multi-hour/multi-day long-form.
3. **Speed of setup** — presets, track templates, token naming, auto take/roll.
4. **Operational awareness (not alarm fatigue)** — leveled INFO/WARNING/CRITICAL with hysteresis + filtering.
5. **Recoverability** — any interruption yields valid, recoverable files.
6. **Interoperability** — clean hand-off to post (BWF/iXML, markers, CSV, daily folders).

1.4 Operating Paradigms (both supported)

- **Take mode** (film/scripted): discrete takes, roll/take/scene, slate-style metadata.
- **Continuous mode** (reality/live/broadcast): long continuous iso record, marker-based logging, time/size/TC rollover.

The UX and data model must serve both; the difference is primarily in counters, naming defaults, and the logging emphasis.

1.5 Operating Modes (state machine)

Setup → Armed → Standby(pre-record buffering) → Recording → Recording(Degraded) ↔ Recording → Stopped → Review

- **Standby/Pre-record:** rolling N-second pre-buffer so record captures audio *before* the press.
- **Recording(Degraded):** capture continues; a non-capture subsystem (mirror/LTC/disk-warning/device-loss) is impaired and surfaced.
- **Review:** spot-check playback, **mutually exclusive with Recording** (isolated engine).
- **Overlays:** **Lock Mode** (block config/routing/device/metadata edits) and **Record Safe Mode** (guard Stop/Quit/Close/Change-project/Change-destination), enforced in the command layer across *all* control surfaces.

1.6 Failure Hierarchy (defines G4)

- **Tier 0 — Sacred (must never stop/corrupt):** input capture, FIFO drain, primary-disk write, crash-valid headers.
- **Tier 1 — Important (degrade, keep recording):** LTC, mirror disk, metering, health monitor, event log.
- **Tier 2 — Peripheral (may pause/fail freely):** remote dashboard, exports, UI cosmetics, control-surface links.

1.7 System Reliability Dependencies (the app owns these)

- Inhibit **App Nap** and **system/display sleep** while Armed/Recording.
- **Destination policy:** local/external physical volumes only; refuse cloud-synced (OneDrive/iCloud/Dropbox), network, and Time-Machine-target volumes.
- **Power:** detect on-battery / low-power / thermal pressure and warn; recommend UPS for the appliance.
- **Time integrity:** treat wall-clock and media-clock as distinct; recommend a disciplined-but-non-stepping system clock during record; handle DST/timezone without breaking naming or midnight logic.
- **OS hygiene (deployment guidance):** disable auto OS updates and Time Machine on the record machine during productions.

1.8 Deployment Posture

Appliance-first: recommend running the target Mac mini M4 as a dedicated recorder appliance (auto-launch, auto-recover after restart, minimal other software, kiosk-like). It also runs as an ordinary app, but appliance mode is the reliability-optimal default.

1.9 Redundancy Stance

- **Near-term (in spec):** mirrored recording to a secondary disk (best-effort; primary never blocked).
- **Operational:** documented "run a second instance/machine as a backup recorder" posture.
- **Deferred:** full A/B chase-sync between instances — acknowledged, not in early scope.

1.10 Personas (de-conflated)

- **Buyer:** production company / facility (in-house first).
- **Production sound mixer / recordist** (Take mode, film/scripted).
- **Audio control-room operator + sound utility** (Continuous mode, reality/live).
- **Broadcast/OB engineer** (high-count Dante, LTC, long runs, disk planning).

- **Rental/facility tech** (builds device/track/session presets).
- **Producer / showrunner** (read-only remote dashboard consumer).
- **Dante / IT network admin** (network + clocking).
- **Post / DIT** (BWF/iXML, daily folders, markers, take logs).

1.11 UX Principles (product-level)

Modern macOS Dark/Light; **colorblind-safe, high-legibility metering** readable at a distance; consistent command model across keyboard/Stream Deck/OSC/HTTP; localization-ready.

1.12 Scope & Non-Goals

- **In scope:** capture, per-track **record-trim gain** + delay + bypass, input metering, markers, isolated spot-check playback, post-record exports, metadata/naming/folders, LTC awareness, health/recovery, mirrored record (near-term), remote dashboard (future).
- **Non-goals:** not a DAW; no editing; no plugins/effects beyond delay+gain; **per-track "gain" is record trim, not a monitor mix bus**; not a routing matrix/mixer.
- **Open product decision (A10):** *live confidence monitoring* (headphone/PFL/solo output) — currently metering-only. **Recommendation:** treat basic confidence monitoring (selectable solo/headphone bus) as in-scope for production-sound credibility, scheduled after MVP; confirm before locking the UX.

Part C — Execution: organize & commit to git

Requires leaving plan mode (commits are blocked in plan mode). On approval:

1. **Move the working tree out of OneDrive** is *not* required for this doc commit, but note the existing repo lives under a OneDrive path; committing a markdown doc is low-risk. (Full relocation remains a Phase-0 task.)
2. Create a docs structure in the repo:
 - docs/architecture/01-product-architecture.md ← hardened Section 1 (Part B).
 - docs/architecture/reviews/01-product-architecture-adversarial-review.md ← Part A (the attack log), for traceability.
 - docs/architecture/README.md ← index noting Sections 2–21 are pending the same treatment.
1. **Logical commit organization** (small, reviewable commits on a feature branch, since `main` is the working branch):
 - Branch: `docs/product-architecture` .
 - Commit 1: `docs: scaffold architecture docs structure (README/index)` .
 - Commit 2: `docs: add hardened product architecture (section 1)` .
 - Commit 3: `docs: add adversarial review log for section 1` .
1. Do **not** push automatically; leave pushing to the user (auto-push.sh exists but pushing is the user's call).

Verification

- `git log --oneline -5` shows the three logical commits on `docs/product-architecture` .
- `git status` clean; the new `docs/architecture/**` files tracked.
- Render the markdown to confirm tables/state-machine read correctly.
- Confirm no edits were made outside `docs/` (the recorder code does not exist yet).

Section 2 — Software Architecture: Adversarial Review & Hardening

Scope: Focus on **Section 2 (Software Architecture)** only. Attack it in isolation from many angles, then harden it.

Context (what Section 2 originally said)

A layered design — Presentation → Application/Command → Engine façade (publishes snapshots) → four engines (Capture RT, Recording, Disk Writer pool, LTC) → mockable Platform abstraction — with four cross-cutting rules: RT/non-RT firewall, one-way snapshot publication, single command choke point, everything mockable. Directionally sound, but several load-bearing details are unspecified or quietly wrong under attack.

Part A — Adversarial Review (by angle)

A1 — Threading model is named but not specified (Critical)

The diagram lists threads but defines no **thread inventory**, **priority policy**, or **non-blocking contract**. Unspecified: who owns each thread, real-time vs high vs normal priority, the rule that the RT thread *never* waits on any other thread, priority-inversion avoidance, and start/stop **shutdown ordering** (a classic use-after-free source). **Fix:** add an explicit Thread Inventory + priority/ownership/lifecycle table and the "RT waits on no one" invariant.

A2 — Two distinct planes are conflated: control vs data (Critical)

"Engine publishes snapshots at ~30 Hz" lumps slow status with the **hot meter/waveform data path**. At 128 tracks the meter plane wants direct lock-free per-track atomics, not a fat status struct copy. **Fix:** split a **Control/Status plane** (commands + status snapshots) from a **Data plane** (audio FIFOs + per-track meter atomics). Different mechanisms, different rates.

A3 — Snapshot mechanism + memory reclamation underspecified (High)

"Double-buffered / atomic seqlock" hand-waves the hard part: seqlock readers spin; pointer-swap double-buffering needs safe reclamation (ABA / who frees the old snapshot). **Fix:** pin a concrete, RT-safe scheme — **triple-buffer with atomic publish index** (no reader spin, no allocation, no free-after-read race) for status; raw atomics for meters.

A4 — How a command reaches the RT thread is missing (High)

"Commands go through the dispatcher" stops at the firewall. Live changes (arm, set gain/delay, bypass) must cross into the RT thread **without locks**. **Fix:** define a **lock-free command FIFO** consumed by the engine at buffer boundaries, plus per-parameter atomics for cheap continuous values (gain/bypass). No control path takes a lock the RT thread could contend.

A5 — No event/error bus; error model undefined (High)

EventLog and HealthMonitor exist as modules, but Section 2 shows **no channel** by which a writer-thread disk-full or an LTC dropout *reaches* them, and no error-handling strategy (exceptions are forbidden in RT). Status snapshots are one-way and continuous; discrete **events** (errors, markers, LTC events, rollover) need their own path. **Fix:** add an explicit **lock-free event bus** (engine → app) for discrete events, and a stated error model: `Result` /error-code returns at boundaries, **no exceptions across the firewall or in RT**.

A6 — No TimeAuthority / Transport component (High)

Sample-clock ↔ wall-clock ↔ timecode mapping is the backbone of BWF `TimeReference`, TC-rollover, markers, and midnight logic — yet "Clock" appears only as the *system* clock in the platform layer. There's no owner of the **sample-count master + sample↔TC↔wall map**. **Fix:** introduce a **Transport / TimeAuthority** component (engine-level) owning the monotonic sample master and the time-domain mapping; LTC/Recording/Marker subsystems consume it.

A7 — Layering inversion: engine writes the RecoveryJournal (Medium)

RecoveryJournal sits in the Application layer, but the **writer (engine layer) must update it during recording** — an engine→application dependency that violates the layering. **Fix:** define journal/event sinks as **interfaces injected downward** (dependency inversion); the engine depends on `IJournalSink` / `IEventSink`, not on app-layer classes.

A8 — Process model: a Tier-2 crash kills Tier-0 (High)

Everything is one process, so a fault in dashboard/export/UI can take down **capture** — directly contradicting "Capture is sacred." **Fix:** state a process-isolation policy — run **exports out-of-process** (ffmpeg child) now, and put the **remote dashboard** (and ideally control-API ingress) behind an **XPC/separate process** so Tier-2 cannot crash Tier-0. Keep MVP simple but reserve the seam.

A9 — FIFO overflow / backpressure policy undefined (High)

"Large buffers absorb disk stalls" is not a policy. What happens when a FIFO actually fills (disk can't keep up)? **Fix:** define the **overflow contract**: RT never blocks; on full FIFO it marks an **xrun event** (counted, surfaced, logged) and follows a stated drop policy; sizing = "seconds of headroom" with the overflow path exercised in tests, not assumed impossible.

A10 — Over-abstraction risk for a solo dev (Medium)

Facade + dispatcher + a dozen services is gold-plate-able. **Fix:** mark the **MVP-essential abstractions** (RT firewall, platform mocks, lock-free FIFO, snapshot/event bus, command FIFO) vs. **start-concrete-extract-later** (full service layer, facade richness). Build simple first; the firewall is the only abstraction that's non-negotiable from day one.

A11 — Single source of truth / change-notification absent (Medium)

Session/Metadata/Naming/Counters all hold state; a metadata change must update the naming preview, etc. No defined ownership or observer model → drift/inconsistency. **Fix:** name a **SessionModel** as single source of truth on the message thread with a change-notification (observer) mechanism feeding dependent services and the UI.

A12 — Schema versioning / migration not architectural (Medium)

JSON sessions/presets and the SQLite journal need **version fields + migration** from v1, or early files break later. **Fix:** add schema-version + forward-migration as a platform/storage concern.

A13 — Control-API trust boundary missing (Medium)

Dashboard is read-only (good), but **OSC/HTTP control is read-write into Tier-0 commands**. No auth/bind-scope/rate-limit/authz stated. **Fix:** define a **network trust boundary**: control ingress is authenticated, LAN-bound, rate-limited, routed through the *same* command choke point (so Record-Safe/Lock apply), and separated from the read-only dashboard surface.

A14 — Startup / recovery / device-acquire sequencing undefined (Medium)

No boot orchestration: detect prior crash → recovery decision → acquire device → restore session → (never auto-arm before recovery is resolved). **Fix:** add a **lifecycle/orchestration** description with explicit ordering and "no auto-arm before recovery decision."

A15 — Engineering observability seam not called out (Medium)

RT-callback duration histograms, FIFO high-watermarks, writer latency feed both tests and HealthMonitor, but aren't an architectural cross-cutting concern. **Fix:** add a **telemetry seam** (lock-free counters/histograms) as the shared source for internal metrics, tests, and user-facing health.

A16 — Stack alignment not stated (Low)

The threading/FIFO mechanisms should map to the chosen stack (JUICE: `AbstractFifo`, `AudioIODeviceCallback`, message thread) and note deviations (custom writer threads vs `ThreadPool`). **Fix:** one note aligning the model to JUCE primitives and where we go custom.

Part B — Hardened Section 2 (Software Architecture)

2.1 Two planes

- **Data plane (hot):** audio input → per-track lock-free SPSC FIFOs → writer pool; per-track **meter atomics** read directly by UI/dashboard. No locks, no allocation, no syscalls.
- **Control/Status plane:** commands in (lock-free command FIFO + parameter atomics) and status out (triple-buffered status snapshot + lock-free **event bus** for discrete events).

2.2 Layers (with corrected dependency direction)

Presentation (GUI + read-only dashboard) → Application (SessionModel = single source of truth, command dispatcher with Record-Safe/Lock gating, services) → **Engine façade** → Engines (Capture RT, **Transport/TimeAuthority**, Recording, Disk Writer pool, LTC) → Platform abstraction (AudioDevice, SystemClock, FileSystem, DiskSpace, Storage[SQLite/JSON, versioned], Net). **Engines depend on injected interfaces (`IEventSink`, `IJournalSink`) — never on app-layer classes.**

2.3 Thread inventory & invariants

Thread	Priority	Owns	Blocking allowed?
Audio callback	Real-time (CoreAudio)	capture, delay/gain, meter+clip taps, FIFO push, consume command FIFO at buffer edge	Never waits on any other thread
Writer pool (2–4)	High (non-RT)	FIFO drain, format, write, header patch, journal sink	May do I/O; never blocks RT
LTC decode	High	LTC tap → decode → events	No
Monitor/health	Normal	telemetry, leveling, snapshots	No
Message/UI	Normal	SessionModel, dispatch, rendering	Yes (off RT)
Net/dashboard	Low	read-only serve; control ingress via dispatcher	Yes (isolated)

Invariants: RT waits on no one; no exceptions in RT or across the firewall; no allocation after Arm; defined thread **start/stop ordering** (acquire device last on start, release first on stop) to prevent use-after-free.

2.4 Cross-cutting concerns (added)

- **Event/error model:** discrete events via lock-free event bus; `Result` /error-codes at boundaries; errors classified by **failure tier** (Sacred/Important/Peripheral from §1.6).
- **Backpressure:** FIFO-full $\Rightarrow$ RT marks counted xrun event + stated drop policy; never blocks.
- **TimeAuthority:** monotonic sample master + sample $\leftrightarrow$ TC $\leftrightarrow$ wall map; sole source for `TimeReference`, rollover, markers, midnight.
- **Telemetry seam:** lock-free counters/histograms feeding tests + health.
- **Persistence:** versioned schemas + migration; `SessionModel` as single source of truth with change-notification.
- **Network trust boundary:** authenticated, LAN-bound, rate-limited control ingress through the one command choke point; read-only dashboard separated.
- **Process isolation:** exports out-of-process now; dashboard/control ingress behind XPC/separate process so Tier-2 can't crash Tier-0.
- **Lifecycle:** boot = detect crash $\rightarrow$ recovery decision $\rightarrow$ acquire device $\rightarrow$ restore session; **no auto-arm before recovery resolved**.

2.5 Solo-dev pragmatism

Non-negotiable from day one: **RT firewall, lock-free FIFO, command FIFO + status/event channels, platform mocks, telemetry seam**. Start concrete and extract later: façade richness, full service layer, XPC isolation (reserve the seam, implement when dashboard/exports land). Don't gold-plate the layering before the capture tracer-bullet proves out.

Verification (Section 2)

- Walkthrough: trace a "disk full at 128ch while recording with LTC" event end-to-end — RT marks xrun $\rightarrow$ event bus $\rightarrow$ health leveling $\rightarrow$ UI/dashboard + EventLog + journal — with **capture never stalling**. The hardened model must express every hop without a lock on the RT thread.

Section 3 — Module Breakdown: Adversarial Review & Hardening

Scope: Focus on **Section 3 (Module Breakdown)** only. Attack the module list in isolation, then re-cut it.

Context (what Section 3 originally said)

A flat 19-row table (`AudioDeviceManager`, `CaptureEngine`, `RecordingEngine`, `DiskWriterPool`, `LTCEngine`, `MeteringService`, `MetadataService`, `NamingEngine`, `CounterService`, `SessionStore`, `RecoveryJournal`, `HealthMonitor`, `EventLog`, `MarkerService`, `ControlSurfaces`, `RemoteDashboard`, `PlaybackEngine`, `ExportService`, `GUI/ViewModels`) with a "responsibility" and a loose "thread context" column. Useful inventory, but it predates the §2 hardening, mixes altitudes, and hides what's reliability-critical.

Part A — Adversarial Review (by angle)

A1 — Modules implied by hardened §1/§2 are missing (Critical)

The breakdown doesn't include components the design now requires: **Transport/TimeAuthority**, **EventBus**, **CommandService** (the one choke point with Record-Safe/Lock gating — currently buried inside "ControlSurfaces"), **SessionModel** (in-memory single source of truth — distinct from the persistence-only "SessionStore"), **TelemetryService**, **PreRecordBuffer** (standby/pre-roll), **Destination/VolumePolicy** (local-only, cloud/network refusal, disk planning), **MirrorWriter**, **PresetService/TemplateService** (lumped into SessionStore), and an **AppCoordinator** (composition root / lifecycle). Optional **MonitoringOutput** if confidence monitoring is approved (§1 open decision). **Fix:** add these as first-class modules.

A2 — Altitude/granularity is inconsistent; criticality invisible (High)

A tiny RT-critical `CaptureEngine` sits in the same flat list as the huge `GUI/ViewModels`, with no grouping by **failure tier** (§1.6) or **plane** (§2.1). You can't see at a glance what must never fail. **Fix:** regroup by **Tier (0/1/2)**, and tag **plane + RT-safety class** per module.

A3 — Overlapping responsibilities / fuzzy ownership (High)

- **Crash-valid headers:** `DiskWriterPool` (header patching) vs `RecoveryJournal` — who owns recoverability?

- **Rollover:** RecordingEngine "orchestration" vs DiskWriterPool "gapless rollover" — decide vs execute unclear.
- **Metering/clip:** the RT *tap* lives in CaptureEngine, aggregation in MeteringService, leveling in HealthMonitor — three owners, fuzzy seam.
- **Filename resolution:** NamingEngine + CounterService + MetadataService all feed it — which one *owns* "produce the final path"?

Fix: state explicit **decide-vs-execute** / **tap-vs-aggregate-vs-level** boundaries per pair.

A4 — No dependency/interaction map (High)

A table can't be validated against §2's "no engine→app dependency" and Tier rules without showing **who talks to whom**. **Fix:** add dependency rules + a short interaction list; assert **no Tier-0 module depends on a Tier-2 module**.

A5 — Thread-context column is vague and omits RT-safety class (Medium)

"Any → serialized", "Message + control" don't say whether a module may be touched from the RT thread. (e.g., EventLog must be **never called from RT** — RT only posts to EventBus.) **Fix:** replace with **thread owner + RT-safety class** (RT-safe / non-RT / never-from-RT).

A6 — Test strategy per module not indicated (Medium)

Pure/deterministic modules (NamingEngine, CounterService, TimeAuthority math) deserve exhaustive unit tests; hardware-bound ones (AudioDeviceManager) need mocks + soak. The table doesn't distinguish. **Fix:** tag each module's **primary test strategy**.

A7 — No lifecycle owner / composition root (Medium)

§2 added boot/recovery sequencing and DI, but no module owns it. **Fix:** add **AppCoordinator** (composition root: builds graph, injects interfaces, owns start/stop ordering, drives the boot→recovery→acquire→restore sequence).

A8 — Process-model decisions not reflected (Medium)

ExportService is shown in-process ("background worker"); §2 hardened it to **out-of-process ffmpeg**. RemoteDashboard likewise XPC-isolated. **Fix:** update both to reflect supervision of isolated child processes.

A9 — ControlSurfaces conflates trust levels (Medium)

Keyboard + Stream Deck + OSC + HTTP in one module spans local input and **network ingress** with very different trust. The §2 network trust boundary has no owner. **Fix:** split **LocalControl** (keyboard / local Stream Deck) from **NetworkControlGateway** (OSC/HTTP: auth, bind-scope, rate-limit); both funnel into **CommandService**.

A10 — Too many modules for a solo dev to scaffold up front (Medium, YAGNI)

~27 modules after additions; MVP (§18) needs a fraction. **Fix:** tag each module with **phase** (MVP / TC / Workflow / Hardening / Later) per §17 so v1 stays small.

A11 — PlaybackEngine device contention undefined (Low/Med)

"Separate audio path" — does it share AudioDeviceManager or open its own device, and who enforces "not while armed"? **Fix:** PlaybackEngine acquires its **own output device**, mutually exclusive with capture; AppCoordinator enforces.

A12 — Naming/suffix convention loose (Low)

SessionStore vs needed SessionModel; mixed Engine/Service/Manager suffixes. **Fix:** convention — **Engine** = RT/near-RT subsystem; **Service** = message-thread logic; **Manager** = resource/lifecycle; **Model** = state.

Part B — Hardened Section 3 (Module Breakdown, regrouped)

Columns: **Tier** (§1.6) · **Plane** (D=data, C=control/status) · **RT class** (RT=runs in callback, NR=non-RT, NFR=never-from-RT) · **Phase**.

Tier 0 — Sacred (capture must never stop/corrupt)

Module	Plane	RT	Phase	Responsibility (boundary)	Test
AudioDeviceManager	C	NR	MVP	device discovery/select/validate, hot-plug, SR/clock monitoring, channel map	mock + hardware
CaptureEngine	D	RT	MVP	callback: delay→gain→bypass→ meter/clip tap →FIFO push; consume command FIFO at buffer edge	RT-guard + soak
PreRecordBuffer	D	RT	TC	rolling pre-roll ring feeding record-start	unit + soak
Transport/TimeAuthority	C	RT-read	MVP	sample master + sample↔TC↔wall map (sole owner)	pure unit
DiskWriterPool	D	NR	MVP	FIFO drain→format(WAV/RF64/BWF)→large writes→ preallocate + crash-valid header patch ; <i>executes</i> rollover	mock-disk + golden
Destination/VolumePolicy	C	NR	MVP	local-only enforcement, cloud/network/TM refusal, disk planning/remaining-time	mock-fs unit

Tier 1 — Important (degrade, keep recording)

Module	Plane	RT	Phase	Responsibility (boundary)	Test
RecordingEngine	C	NR	MVP	arm/start/stop, segment lifecycle, decides rollover (writer executes), sample↔TC stamping	mock + soak
LTCEngine	C	NFR	TC	decode (liblbc), TC tracking, dropout/discontinuity/rate/midnight events	synthetic corpus
MeteringService	C	NFR	MVP	aggregate RT meter atomics → UI/dashboard rate (CaptureEngine taps, HealthMonitor levels)	unit
HealthMonitor	C	NFR	Hardening	sample CPU/RAM/disk/FIFO/device/LTC/clip; level w/ hysteresis	unit (fake metrics)
RecoveryJournal	C	NFR	Hardening	durable WAL of files/segments/markers; owns <i>journal</i> , writer owns <i>headers</i>	crash/kill tests
EventBus	C	RT→NR	MVP	lock-free engine→app discrete events	unit
EventLog	C	NFR	Hardening	append-only operational log (consumes EventBus)	unit
TelemetryService	C	RT→NR	MVP	lock-free counters/histograms (callback duration, FIFO HWM, writer latency)	unit
MirrorWriter	D	NR	Later	best-effort secondary write; failure never blocks primary	mock-disk

Tier 2 — Peripheral (may pause/fail freely; ideally isolated)

Module	Plane	RT	Phase	Responsibility	Test
SessionModel	C	NFR	MVP	in-memory single source of truth + change-notification	unit
SessionStore	C	NFR	Workflow	persistence (JSON, versioned + migration)	unit
PresetService / TemplateService	C	NFR	Workflow	session presets + track templates	unit
MetadataService	C	NFR	Workflow	field model, templates, live-edit rules, BWF/iXML mapping	unit
NamingEngine	C	NFR	Workflow	owns final-path resolution from tokens (pulls metadata+counters+TC); sanitize/dedupe/preview	pure unit/property
CounterService	C	NFR	Workflow	roll/take/segment + increment rules	pure unit
MarkerService	C	NFR	Hardening	marker capture (TC/ts/note), CSV/PDF export	unit
CommandService	C	NFR	MVP	the choke point: Record-Safe/Lock gating; all surfaces funnel here	unit
LocalControl	C	NFR	Hardening	keyboard / local Stream Deck → CommandService	unit
NetworkControlGateway	C	NFR	Later	OSC/HTTP ingress: auth, bind-scope, rate-limit → CommandService	unit + fuzz
RemoteDashboard	C	NFR	Later	read-only; XPC/isolated; serves snapshots	integration
PlaybackEngine	D	NR	Later	isolated spot-check; own output device , mutually exclusive w/ capture	integration
ExportService	C	NFR	Later	supervises out-of-process ffmpeg from originals	integration
GUI/ViewModels	C	NFR	MVP	bind to snapshots/events; send Commands	snapshot/UI
AppCoordinator	C	NFR	MVP	composition root: DI wiring, start/stop ordering, boot→recovery→acquire→restore	integration

Dependency rules

- **No Tier-0 module depends on a Tier-1 or Tier-2 module.** Tier-0 emits via EventBus/Telemetry interfaces only (`IEventSink` , `ITelemetrySink` , `IJournalSink`) — injected, never concrete app types.
- RT-class **RT/NFR** modules are **never called from the audio thread** except those marked RT; cross-thread contact is FIFO/atomic/EventBus only.
- All control surfaces (Local, Network, GUI) reach the engine **only** through CommandService.

Verification (Section 3)

- Cross-check every §2 cross-cutting concern maps to an owning module (TimeAuthority↔Transport, event bus↔EventBus, trust boundary↔NetworkControlGateway, lifecycle↔AppCoordinator, backpressure↔CaptureEngine+Telemetry).
- Confirm the MVP-phase set is small (the Tier-0 row + SessionModel/CommandService/GUI/AppCoordinator) and nothing MVP-tagged depends on a Later-tagged module.